ARQUITECTURAS

Arquitectura en capas

¿Qué es? Es una forma de organizar un programa dividiéndolo en capas, una encima de otra. Cada capa tiene una tarea propia (mostrar pantallas, aplicar las reglas del negocio, manejar los datos) y solo se comunica con la capa vecina.

Para qué sirve? Sirve para tener el código ordenado, hacer cambios en una capa sin dañar las demás, probar cada parte por separado y trabajar en equipo sin estorbarse.

Singleton y Factory Method

Singleton

- Definición: Una sola instancia.
- Función: Compartir un recurso.
- Ejemplo: Conexión a una base de datos.

Factory Method

- Definición: Crea objetos.
- Función: Organizar la creación de objetos.
- Ejemplo: Crear usuarios de distintos tipos.

Decorator, Proxy y Bridge

Decorator: Añade funcionalidades a un objeto sin modificar su estructura original.
Ejemplo: Agregar ingredientes extra a un plato.

- Proxy: Controla el acceso a un objeto.

Ejemplo: Verificar permisos antes de acceder a información.

- Bridge: Separa la abstracción de su implementación para que ambas puedan cambiar independientemente.

Ejemplo: Usar diferentes tipos de pedidos con distintos sistemas de notificación.

Arquitectura MVC

Es una forma de organizar el código de una aplicación en tres partes:

- Modelo: Maneja los datos.
- Vista: Muestra la información.
- Controlador: Gestiona las acciones del usuario.

Abstract Factory, Builder y Prototype

Abstract Factory: Permite crear familias de objetos relacionados sin especificar sus clases concretas.
Builder: Permite construir objetos complejos paso a paso.
Prototype: Permite crear nuevos objetos copiando uno existente.
Ejemplos:

- Abstract Factory: Crear botones y menús de diferentes estilos.
- Builder: Construir un pedido con varios productos.
- Prototype: Copiar un producto existente para crear otro similar.

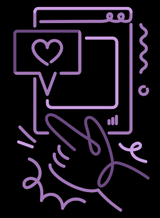
Arquitectura monolítica

Es una arquitectura de software en la que todos los módulos de una aplicación, como la interfaz, la lógica de negocio y el acceso a la base de datos, forman parte de un mismo sistema y se ejecutan como una sola unidad.

Arquitectura cliente-servidor

Es una arquitectura donde el cliente solicita servicios o información y el servidor procesa las solicitudes y devuelve una respuesta.
Características:

- Cliente: Interfaz que usa el usuario.
- Servidor: Procesa las solicitudes.
- Comunicación: Intercambio de datos por una red



Adapter y Facade

Adapter: Permite que dos sistemas incompatibles puedan trabajar juntos.
Ejemplo: Conectar una aplicación con una API diferente.

- Facade: Simplifica el acceso a un sistema complejo mediante una interfaz única.

Ejemplo: Un botón que procesa un pedido y coordina el pago y el inventario

ARQUITECTURAS

Arquitectura de microservicios

Es un enfoque de desarrollo de software donde una aplicación completa se divide en un conjunto de pequeños servicios independientes, autónomos y especializados

Composite y Flyweight

El patrón Composite se utiliza cuando necesitas representar estructuras jerárquicas en forma de árbol (padres e hijos) y quieres que el código cliente trate a los objetos

El patrón Flyweight se utiliza cuando necesitas crear una enorme cantidad de objetos similares y el programa se queda sin memoria RAM para manejarlos.

- El problema que resuelve: Reduce el consumo de memoria dividiendo los datos de un objeto en dos partes

Strategy y State

se utiliza para definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. Permite que el algoritmo varíe independientemente

permite que un objeto altere su comportamiento cuando su estado interno cambia. El objeto parecerá haber cambiado de clase.



Arquitectura hexagonal

Es la separación de responsabilidades. Su estructura se divide en tres partes bien definidas, donde la regla de oro es que el exterior depende del interior, pero el interior jamás sabe nada del exterior.

lases que representan el negocio (ej. Usuario, Pedido, Factura) y los servicios de dominio que calculan reglas (ej. CalculadorImpuestos).

Arquitectura orientada a servicios (SOA)

modelo de diseño de software en el que las aplicaciones se construyen mediante componentes o servicios independientes que se comunican entre sí a través de una red

Observer y Mediator

Un objeto cambia (Sujeto) e inmediatamente "grita" al viento. Todos los que estén anotados en su lista (Observadores) escuchan el grito mediante su método update().

Los objetos no gritan ni se hablan entre sí. Cuando un objeto hace algo, va en privado con el Mediador y le dice: "Hice esto". El Mediador, que tiene el cerebro y las reglas de todo el sistema, decide a qué otros objetos llamar y qué ordenarles